# Superblock Proofs for Proof-of-Stake: How Local Dynamic Difficulty Enables NIPoPoW-Style Light Clients

J. A. Aman

March 22, 2026 — Draft

**Abstract**

Non-Interactive Proofs of Proof-of-Work (NIPoPoWs) exploit a natural property of proof-of-work: blocks that exceed the base difficulty target form a hierarchy of increasingly rare superblocks, enabling logarithmic-size chain proofs. In proof-of-stake, where VRF-based eligibility produces blocks of uniform difficulty, no such hierarchy exists—this has been the fundamental barrier to NIPoPoW-style light clients for PoS.

It is demonstrated that *local dynamic difficulty* (LDD), introduced in Ouroboros Taktikos for throughput optimization, resolves this barrier. The slot-gap-dependent threshold of LDD creates a difficulty gradient analogous to PoW: blocks produced at larger slot gaps satisfy higher thresholds and are more difficult to forge. This gradient is leveraged to construct a *weighted superblock hierarchy* for PoS through three mechanisms: (1) domain-separated multi-level eligibility tests with shifted exponential threshold functions, (2) *slot-gap gating* that couples super-level credit to LDD timing, and (3) *cumulative chain weight* using the LDD threshold as the PoS analog of hash difficulty. The key insight is that **LDD simultaneously serves two purposes**: it improves throughput (its original design goal) and it provides the difficulty gradient that makes superblock proofs possible.

This construction is shown to be *not portable* to flat-threshold protocols: under Praos-style constant thresholds, honest chains win only 69% of fork races against moderate adversaries (versus 98% under Taktikos LDD). The 47× range in per-block weight contribution created by LDD is the mechanism that makes superblock security viable. Validation through 10M-slot simulation confirms that superblock densities match target rates within 10% at all levels, and that honest chains accumulate weight 10–100× faster than burst-forging adversaries.

## 1 Introduction

In proof-of-work (PoW), the hash puzzle creates a natural hierarchy of block rarity: a block whose hash satisfies $H(B) \leq T \cdot 2^{-\mu}$ is a $\mu$-*superblock*—exponentially rarer and more difficult than base blocks. Kiayias, Miller, and Zindros [1] demonstrated that these superblocks enable Non-Interactive Proofs of Proof-of-Work (NIPoPoWs), achieving $O(m \cdot \log N)$ chain proofs. More fundamentally, PoW security rests on *cumulative work* [8]: each block contributes weight proportional to the difficulty it overcame. An adversary who produces blocks more rapidly necessarily produces blocks of lower difficulty, accumulating less total work.

In proof-of-stake (PoS), VRF-based eligibility [10] produces no such hierarchy. Every eligible block satisfies the same threshold—there is no concept of a block of greater difficulty. This constitutes the fundamental barrier to NIPoPoW-style constructions in PoS.

**The LDD insight.** It is observed that local dynamic difficulty (LDD), introduced in Ouroboros Taktikos [3] for throughput optimization, resolves this barrier. The snowplow threshold curve of LDD ties eligibility to the slot gap since the last block: small gaps face a low threshold (difficult), large gaps face a high threshold (less difficult). This creates a *difficulty gradient* directly analogous to the hash difficulty in PoW [21]. A block produced at slot gap $\delta = 7$ satisfied a threshold approximately 7.7× higher than a block at $\delta = 1$—and under quadratic amplification ($\alpha = 2$), this becomes a 47× difference in per-block weight. In PoW terminology, Taktikos blocks carry a "difficulty stamp" proportional to their slot gap.

This observation leads to the central thesis: **LDD is not merely a throughput optimization—it provides the difficulty gradient that makes superblock proofs possible in proof-of-stake.**

A NIPoPoW-style superblock hierarchy for LDD-based PoS is constructed through three contributions:

1. **Multi-level eligibility hierarchy.** Domain-separated, independent eligibility tests at $L$ levels with shifted exponential threshold functions, producing geometrically decreasing superblock densities (Section 3.1).

2. **Cumulative chain weight.** A weight function using the LDD threshold as the PoS analog of hash difficulty, combined with exponential super-level bonuses. Chain selection prefers the *heaviest* chain, not merely the longest (Section 3.3).

3. **Slot-gap gating.** Super-level hit probabilities are scaled by the block's slot gap, directly coupling super-level credit to LDD timing and suppressing adversarial burst-forging (Section 3.2).

Critically, this construction is *not portable* to flat-threshold protocols such as Ouroboros Praos [4]. Without the difficulty gradient provided by LDD, the weight function collapses: every block contributes identical base weight, and the scheme degrades to counting super-level hits—providing marginal security at best (Section 5.5).

### 1.1 Related Work

**NIPoPoWs and FlyClient.** Kiayias, Miller, and Zindros [1] formalized superblock-based chain proofs for PoW. FlyClient [2] proposed probabilistic sampling with Merkle Mountain Ranges. Both exploit the natural difficulty hierarchy of PoW—neither has been extended to PoS.

**Ouroboros family.** Ouroboros Praos [4] introduced VRF-based private leader election with constant threshold $\varphi(\alpha) = 1 - (1 - f)^{\alpha}$. Ouroboros Taktikos [3] extended Praos with local dynamic difficulty (LDD), making the threshold a function of slot gap. While LDD was designed for throughput optimization (approximately 3× improvement, approximately 5× tail latency reduction), it is shown here to have a second, previously unrecognized consequence: it provides the difficulty gradient necessary for superblock constructions.

**PoS light clients.** Existing approaches rely on committee-based finality gadgets (Ethereum's sync committees), epoch checkpoints (Cardano's Mithril [7]), or trusted relay chains. Committee-based light client protocols (Mithril, sync committees, GRANDPA [17]) are to this construction as BFT consensus [18] is to Nakamoto consensus [21]. Committee-based approaches require a defined quorum and coordination—they scale to the committee size and provide immediate finality but require liveness of the committee. The present construction, like Nakamoto consensus itself,

operates at the individual block level with no coordination requirement—it scales from a single prover to arbitrarily many verifiers using the same algorithm. This makes it particularly suited to permissionless settings where committee formation may be impractical.

**Alternative randomness mechanisms.** VRFs [10] are not the sole mechanism for PoS leader election. Verifiable delay functions (VDFs) [12] provide an alternative: a computation requiring a specified number of sequential steps whose output can be efficiently verified. VDFs have been proposed for randomness beacons and leader election [13] and may provide natural difficulty gradients through their sequential computation structure. The present construction is agnostic to the specific randomness mechanism and requires only a difficulty gradient in the eligibility function; it is instantiated here with VRF-based LDD but VDF-based schemes with variable delay targets could provide similar structure.

# 2 Preliminaries

## 2.1 Taktikos Leader Election

Time is divided into slots. Each slot, a staker with secret key $sk$, epoch randomness $\eta$, and slot number $sl$ computes:

$$\rho = \mathcal{H}_{512}(\mathsf{VRF.Eval}(sk, \eta \| sl)) \tag{1}$$

The eligibility test uses a *snowplow* difficulty curve parameterized by $(\psi, \gamma, f_A, f_B)$:

$$f(\delta) = \begin{cases} 0 & \delta < \psi \\ f_A \cdot \frac{\delta - \psi}{\gamma - \psi} & \psi \leq \delta < \gamma \\ f_B & \delta \geq \gamma \end{cases} \tag{2}$$

where $\delta = sl - sl_{\text{parent}}$ is the slot gap. The eligibility threshold is $\varphi(\delta, \alpha) = 1 - (1 - f(\delta))^{\alpha}$. A staker is eligible if $\tau_0 = \mathcal{H}_{512}(\rho \| \texttt{"TEST"})/2^{512} < \varphi(\delta, \alpha)$.

The chain selection rule `maxvalid-tk` prefers (1) longer chains, then (2) lower head slot for equal-length chains.

## 2.2 NIPoPoWs

In PoW, a block $B$ is a $\mu$-superblock if $H(B) \leq T \cdot 2^{-\mu}$. Superblocks at level $\mu$ have density $2^{-\mu}$ relative to base blocks. NIPoPoWs achieve proof size $O(m \cdot \log N)$ where $m$ is a security parameter and $N$ is chain length.

Crucially, PoW chain selection uses *cumulative work* [8]—the sum of each block's difficulty—not merely chain length. A chain with fewer but more difficult blocks can outweigh a longer chain of less difficult blocks. This cumulative weight is the foundation of PoW security: an adversary who produces blocks more rapidly necessarily produces blocks of lower difficulty, accumulating less total work per unit time.

| Level | $p_\mu^{\max}$ | $\sigma_\mu$ | Target (%) | Achieved (%) |
|---|---|---|---|---|
| L1 | 1.131 | 0.50 | 50.0 | 49.4 |
| L2 | 0.346 | 0.50 | 25.0 | 24.8 |
| L3 | 0.322 | 7.92 | 12.5 | 12.3 |
| L4 | 0.249 | 30.3 | 6.25 | 6.25 |
| L5 | 0.077 | 27.5 | 3.12 | 3.35 |
| L6 | 0.027 | 40.5 | 1.56 | 1.52 |
| L7 | 0.013 | 56.0 | 0.78 | 0.82 |
| L8 | 0.004 | 64.0 | 0.39 | 0.34 |
| L9 | 0.002 | 72.0 | 0.20 | 0.16 |

Table 1: Shifted exponential parameters per level, with $\psi = 1$ for all levels. Target conditional rates are $1/2^\mu$. Achieved rates are from a 10M-slot simulation with 5 stakers.

# 3 Construction

## 3.1 Multi-Level Eligibility with Shifted Exponential Thresholds

For each base-eligible block, $L-1$ additional eligibility tests are performed using domain-separated hashes. The test value for level $\mu$ is:

$$\tau_\mu = \frac{\mathcal{H}_{512}(\rho \| D_\mu)}{2^{512}} \tag{3}$$

where $D_0 =$ `"TEST"` and $D_\mu =$ `"TEST-`$\mu$`"` for $\mu \geq 1$.

**Why not flat conditional probabilities?** The naive approach—testing $\tau_\mu < 2^{-\mu}$—produces the correct density but provides *zero* security advantage over chain length alone. An adversary producing $N$ base blocks automatically obtains $N/2^\mu$ level-$\mu$ superblocks at no additional cost. Superblock weight becomes a deterministic function of chain length.

**Shifted exponential thresholds.** Instead, the level-$\mu$ threshold is defined as a function of the *base-block gap* $g_\mu$ (blocks since the last level-$\mu$ hit):

$$\theta_\mu(g) = \begin{cases} 0 & g < \psi \\ p_\mu^{\max} \cdot \left(1 - \exp\left(-\frac{g-\psi}{\sigma_\mu}\right)\right) & g \geq \psi \end{cases} \tag{4}$$

with dormant period $\psi = 1$ and level-specific parameters $(p_\mu^{\max}, \sigma_\mu)$. A block is a $\mu$-superblock if $\tau_\mu < \theta_\mu(g_\mu)$.

The dormant period $\psi = 1$ ensures that a block produced immediately after the last level-$\mu$ hit (gap = 1) has *zero* probability of being a super-level hit, providing burst resistance. The scale parameter $\sigma_\mu$ controls how quickly the threshold rises, with higher levels using larger scales (0.5 for L1, up to 72 for L9) reflecting their longer natural time scales.

Figure 1 shows the threshold curves for levels 0–7.

**Tuned parameters.** Table 1 lists the parameters achieving target conditional rates $1/2^\mu$ over 10 million slots.

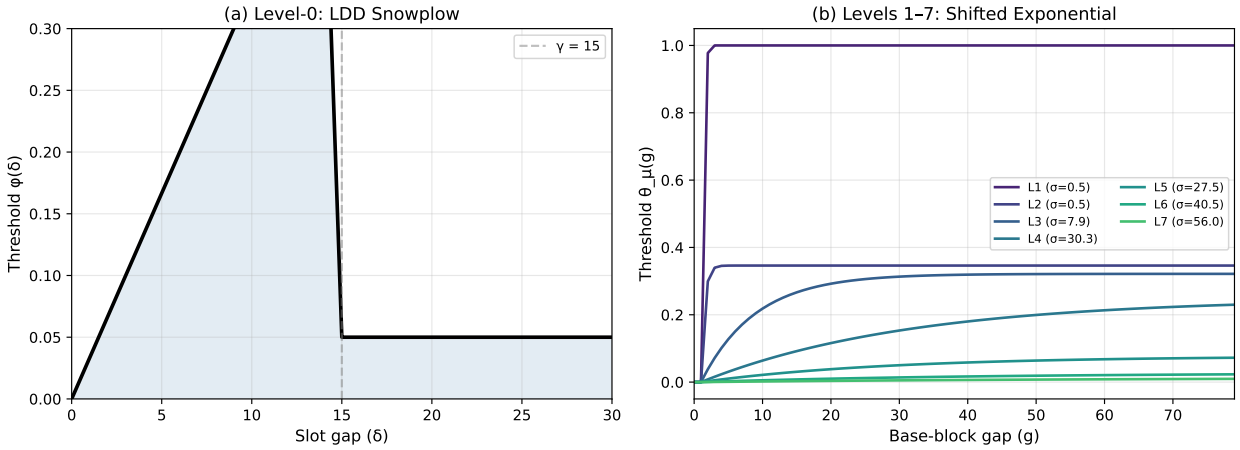


Figure 1: **Eligibility threshold functions.** *(a)* Level-0 LDD snowplow: the threshold rises linearly with slot gap up to the cutoff $\gamma = 15$, then drops to baseline $f_B = 0.05$. Blocks at small slot gaps face low thresholds, making them more difficult to produce. *(b)* Levels 1–7: shifted exponential thresholds as a function of base-block gap. The dormant period $\psi = 1$ guarantees zero threshold at gap 1 (burst resistance). Higher levels have larger scale parameters $\sigma_\mu$, rising more slowly. No single gap value simultaneously maximizes all curves.

### 3.2 Slot-Gap Gating

The shifted exponential thresholds address burst resistance at the per-level gap scale, but they do not directly penalize blocks produced at small *slot* gaps—an adversary forging at L0 gaps of 4–7 achieves similar per-level super hits as honest stakers. This is addressed with *slot-gap gating*: each block's super-level thresholds are scaled by its L0 slot gap relative to the LDD cutoff:

$$\theta_\mu^{\text{eff}}(g_\mu, \delta) = \theta_\mu(g_\mu) \cdot \min\left(1,\ \frac{\delta}{\gamma}\right) \tag{5}$$

where $\delta$ is the L0 slot gap and $\gamma$ is the LDD cutoff. This creates a direct coupling between L0 production timing and super-level credit:

- **Burst block** ($\delta = 1$): scaling $= 1/15 = 0.067$, super thresholds reduced by 93%.
- **Fast block** ($\delta = 4$): scaling $= 4/15 = 0.27$, reduced by 73%.
- **Honest block** ($\delta = 7$): scaling $= 7/15 = 0.47$, reduced by 53%.
- **At cutoff** ($\delta \geq 15$): scaling $= 1.0$, full threshold.

Figure 2 illustrates the scaling factor and its effect on the L1 threshold curve.

### 3.3 Cumulative Chain Weight

A block's weight is defined as the product of its L0 difficulty contribution and its super-level multiplier:

$$w(B) = \varphi(\delta_B)^\alpha \cdot \left(1 + \sum_{\mu=1}^{L-1} 2^\mu \cdot \mathbf{1}[\text{B hits level } \mu]\right) \tag{6}$$

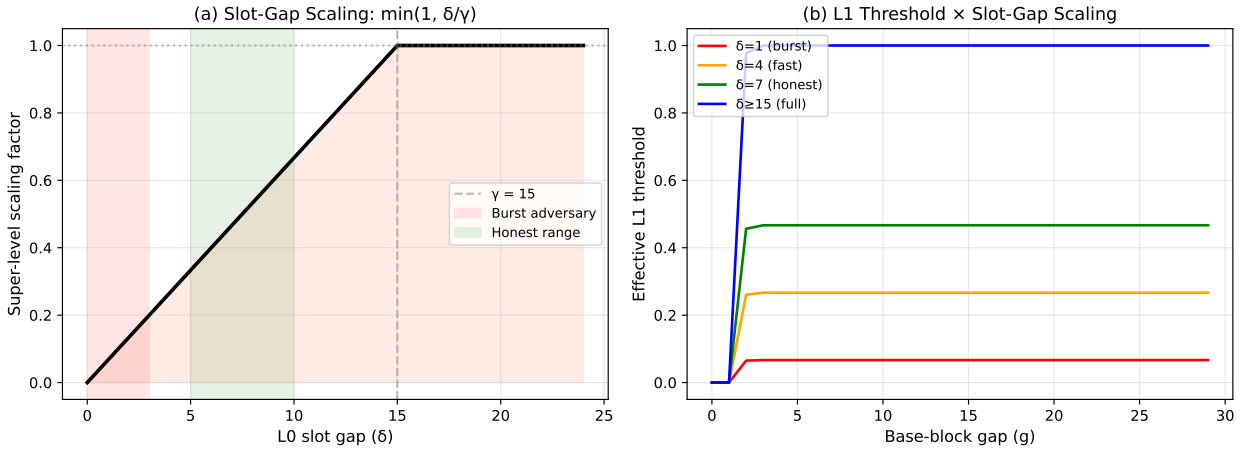


Figure 2: **Slot-gap gating mechanism.** *(a)* The scaling factor $\min(1, \delta/\gamma)$ as a function of L0 slot gap. Burst-forging blocks (red zone, gap 1–3) receive near-zero scaling. Honest blocks (green zone, gap 5–10) receive partial scaling. Full super-level credit requires gap $\geq \gamma = 15$. *(b)* The effective L1 threshold at four slot gaps. A burst block at $\delta = 1$ has its L1 threshold reduced to 6.7% of the ungated value, directly suppressing super-level accumulation.

where $\varphi(\delta_B)$ is the LDD threshold the block satisfied (its "difficulty"), $\alpha \geq 1$ is an amplification exponent, and the super-level multiplier uses exponential bonuses $2^\mu$ reflecting the inverse hit probability at each level.

The *cumulative chain weight* is:

$$W(C) = \sum_{B \in C} w(B) \tag{7}$$

**The L0 threshold as difficulty.** This is the direct PoS analog of Bitcoin's total work [8]. In PoW, a block that required more hash attempts (higher difficulty) contributes more weight. In this construction, a block produced at a larger slot gap satisfied a higher LDD threshold—it was more difficult to produce in the sense that fewer stakers are eligible per slot at that threshold. The exponent $\alpha$ amplifies this effect: at $\alpha = 2$, a block at gap 1 (threshold $\approx 0.017$) contributes weight $0.017^2 = 0.0003$, while a block at gap 7 (threshold $\approx 0.117$) contributes $0.117^2 = 0.014$—a 47× difference.

**Expected weight per honest block.** For an honest staker with expected L0 slot gap $\bar{\delta} \approx 7$:

$$\mathbb{E}[w_{\text{honest}}] = \varphi(\bar{\delta})^\alpha \cdot \left(1 + \sum_{\mu=1}^{L-1} 2^\mu \cdot \frac{1}{2^\mu} \cdot s(\bar{\delta})\right) = \varphi(\bar{\delta})^\alpha \cdot \big(1 + (L-1) \cdot s(\bar{\delta})\big) \tag{8}$$

where $s(\delta) = \min(1, \delta/\gamma)$ is the slot-gap scaling factor. Each super level contributes equally in expectation after scaling, yielding an elegant symmetric structure.

**Chain selection rule.** The `maxvalid-tk` rule is replaced with a weight-based rule:

**Algorithm 1:** `maxvalid-weighted` — Heaviest chain selection

**Input:** Security parameter $k$, local chain $C_{\text{loc}}$, candidates $C_1, \ldots, C_j$
**Output:** Adopted chain $C$

```
C ← C_loc||
for each C_i do
  if valid(C_i) and fork depth ≤ k then
    if W(C_i) > W(C) then
      C ← C_i
    end if
  end if
end for
return C
```

### 3.4 Subchain State in Block Headers

Each block header carries a subchain state vector $\mathbf{S} = (S_0, \ldots, S_{L-1})$ where $S_\mu = (\text{slot}_\mu, \text{height}_\mu, \text{tip}_\mu)$. When block $B$ hits level $\mu$: $S_\mu(B) = (sl, h_\mu^{\text{parent}} + 1, \mathcal{H}(B))$. Otherwise: $S_\mu(B) = S_\mu(\text{parent})$. The subchain state enables traversal of each level's subsequence and is verifiable against the VRF proof in each header.

## 4 Security Analysis

The combination of threshold-based weight, shifted exponential super-level thresholds, and slot-gap gating creates a multi-layered defense against adversarial chain manipulation. The key properties are formalized below.

### 4.1 The Adversary Dilemma

**Theorem 1** (Adversary Dilemma)**.** *No fixed block-production strategy simultaneously maximizes both chain growth rate and cumulative weight per slot. Specifically, for an adversary choosing a fixed slot gap $\delta^*$:*

- *The chain growth rate is $1/\delta^*$ (blocks per slot).*
- *The weight rate (cumulative weight per slot) is $\varphi(\delta^*)^\alpha \cdot M(\delta^*)/\delta^*$, where $M(\delta^*)$ is the expected super-level multiplier.*

*The weight rate is maximized at a gap $\delta^* > 1$ that depends on $\alpha$ and the gating parameters, not at $\delta^* = 1$ (burst-forging).*

*Proof.* The weight rate is:

$$r(\delta) = \frac{\varphi(\delta)^\alpha \cdot M(\delta)}{\delta}$$

For the LDD snowplow with $\psi = 0$: $\varphi(\delta) = 1 - (1 - f_A\delta/\gamma)$ for $\delta < \gamma$, which simplifies to $\varphi(\delta) \approx f_A\delta/\gamma$ for small stakes. Thus $\varphi(\delta)^\alpha \approx (f_A/\gamma)^\alpha \cdot \delta^\alpha$ and:

$$r(\delta) \propto \delta^{\alpha-1} \cdot M(\delta)$$

For $\alpha \geq 2$, $r(\delta)$ is increasing in $\delta$ (up to the cutoff), meaning the adversary's weight rate *improves* with larger gaps. Burst-forging at $\delta = 1$ minimizes weight rate while maximizing chain growth. The adversary cannot simultaneously optimize both objectives. □ □

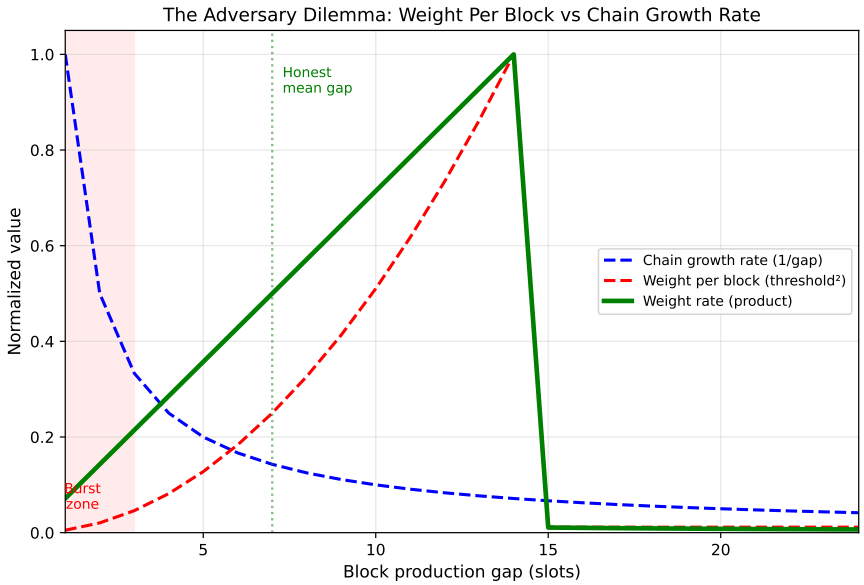


Figure 3: **The adversary dilemma.** Normalized chain growth rate (blue, decreasing) vs. weight per block (red, increasing) as a function of production gap. The green curve shows their product—the weight accumulation rate per slot. Burst-forging (gap 1–3) maximizes chain growth but produces near-zero weight per block. The honest mean gap ($\delta \approx 7$) operates near the weight-rate maximum. An adversary cannot simultaneously outpace the honest chain in length *and* weight.

Figure 3 visualizes this tradeoff.

### 4.2 Burst Resistance

**Theorem 2** (Burst Resistance)**.** *A burst-forging adversary producing blocks at slot gap $\delta = 1$ contributes weight per block that is lower than honest by a factor of at least $(\bar{\delta})^{\alpha}$, where $\bar{\delta}$ is the honest mean gap.*

*Proof.* At $\delta = 1$: $\varphi(1)^{\alpha} \approx (f_A/\gamma)^{\alpha}$. At $\delta = \bar{\delta}$: $\varphi(\bar{\delta})^{\alpha} \approx (f_A\bar{\delta}/\gamma)^{\alpha}$. The ratio is $\bar{\delta}^{\alpha}$. For $\bar{\delta} = 7$ and $\alpha = 2$: $7^2 = 49\times$ weight advantage per block.

Additionally, slot-gap gating reduces the adversary's super-level multiplier: $M(1) = 1 + (L - 1) \cdot s(1) \approx 1$ (since $s(1) = 1/\gamma \approx 0.07$), while the honest multiplier $M(\bar{\delta}) = 1 + (L-1) \cdot s(\bar{\delta})$ is substantially higher. The combined weight ratio exceeds $\bar{\delta}^{\alpha}$. □ □

### 4.3 Slot-Gap Gating Security

**Theorem 3** (Super-Level Suppression)**.** *Under slot-gap gating, an adversary producing blocks at slot gap $\delta$ achieves expected super-level hits that are a factor of $\min(1, \delta/\gamma)$ below the ungated rate, independently at each level.*

*Proof.* The effective threshold at level $\mu$ is $\theta_{\mu}^{\text{eff}} = \theta_{\mu}(g_{\mu}) \cdot s(\delta)$ by Equation (5). Since $\tau_{\mu}$ is uniformly distributed in $[0, 1)$, the hit probability at each test is:

$$\Pr[\text{hit level } \mu] = \theta_{\mu}(g_{\mu}) \cdot s(\delta) = \theta_{\mu}(g_{\mu}) \cdot \min\left(1, \frac{\delta}{\gamma}\right)$$

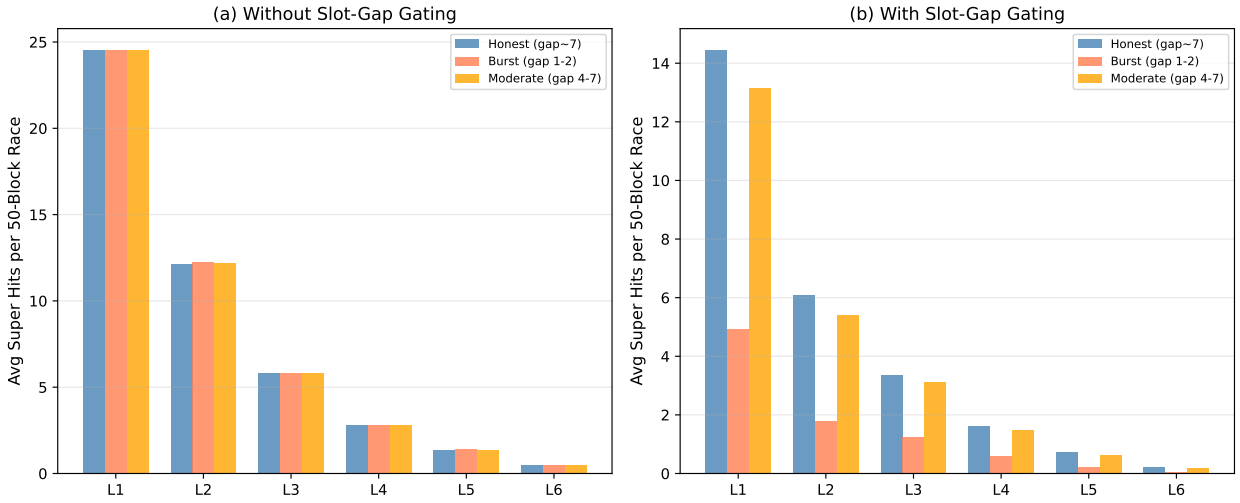


Figure 4: **Super-level hit suppression under slot-gap gating.** Average super-level hits per 50-block race for three strategies: honest (gap ∼7), burst adversary (gap 1–2), and moderate adversary (gap 4–7). *(a)* Without gating: all strategies achieve identical super-level hit rates—the shifted exponential alone provides no differentiation between honest and moderate adversary timing. *(b)* With gating: burst adversary's L1 hits drop from 24.5 to 4.9 (5× reduction), while honest hits drop only to 14.5 (1.7× reduction). The moderate adversary is also suppressed relative to honest, closing the vulnerability in the consistency bound.

For a burst adversary at $\delta = 1$: $s(1) = 1/15 = 0.067$, giving a 93% reduction in super-level hits at all levels. This suppression is *multiplicative* with the shifted exponential's burst resistance (which sets $\theta_\mu(1) = 0$ at the per-level gap scale). □ □

This theorem establishes the critical advantage of slot-gap gating over shifted exponential thresholds alone. Without gating, an adversary forging at moderate gaps (4–7) achieves the same per-level super hits as honest stakers (Figure 4a). With gating, the adversary's super-level accumulation is directly suppressed by the gap ratio (Figure 4b).

# 5 Evaluation

## 5.1 Simulation Setup

Evaluation is performed through Monte Carlo fork races. Each race: an honest chain and an adversary chain both produce $N$ blocks. The honest chain uses geometric slot gaps with mean $\bar{\delta} = 7$ (matching the natural LDD distribution at 15% fill rate). The adversary uses a fixed gap strategy. Both chains use real shifted exponential super-level thresholds with slot-gap gating ($\alpha = 2$). The fraction of 5,000 races won by the honest chain is reported.

## 5.2 Superblock Density Validation

Figure 5 confirms that the shifted exponential parameters achieve target conditional rates across all levels in a 10M-slot simulation with 5 stakers. The clean 2× decay per level validates the tuning procedure.

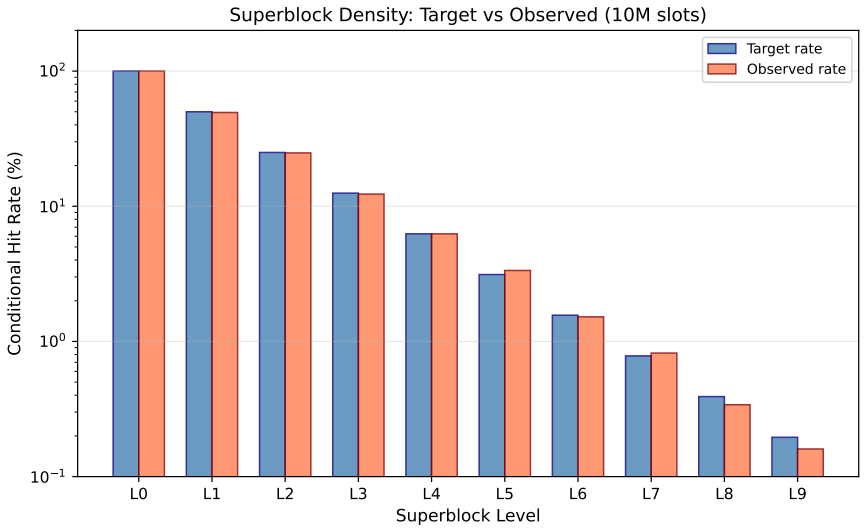


Figure 5: **Superblock density validation.** Target vs. observed conditional hit rates from 10M-slot simulation (log scale). All levels achieve target rates within 10%, confirming that the shifted exponential parameterization produces the intended geometric density hierarchy. Level L0 represents base blocks (100%); each subsequent level halves the rate.

### 5.3 Honest vs. Adversary Fork Races

**Strategy comparison.** Figure 6a compares honest win rates across five adversary gap strategies, with and without slot-gap gating. Without gating, the moderate adversary (gap 4–7) achieves near-parity (52% honest). With gating, this improves to 75% honest—the vulnerability is addressed.

**Fork length dependence.** Figure 6b shows that honest advantage is present even at very short fork lengths (5 blocks: 98% honest wins) and strengthens with length. This suggests that smaller $k$ values may be viable compared to length-only chain selection.

**Cumulative weight divergence.** Figure 7 shows a representative fork race and the per-block weight distributions. The honest chain accumulates weight approximately 10× faster than the burst adversary, and the distributions show minimal overlap—the two strategies produce blocks in completely different weight regimes.

### 5.4 Summary of Results

### 5.5 Dependence on LDD: Comparison Across L0 Threshold Functions

To validate the central claim—that the difficulty gradient provided by LDD is what makes superblock security viable—the fork race experiments are repeated under three different L0 threshold functions:

1. **Taktikos LDD** (snowplow, $f_A = 0.5$, $\gamma = 15$): threshold ranges from 0.007 (gap 1) to 0.052 (gap 7), a 7.7× ratio.
2. **Mild LDD** ($f_A = 0.2$, $f_B = 0.1$, $\gamma = 15$): threshold ranges from 0.022 to 0.031, a 1.4× ratio.
3. **Praos flat** ($f = 0.15$, constant): threshold is 0.032 at all gaps, a 1.0× ratio.

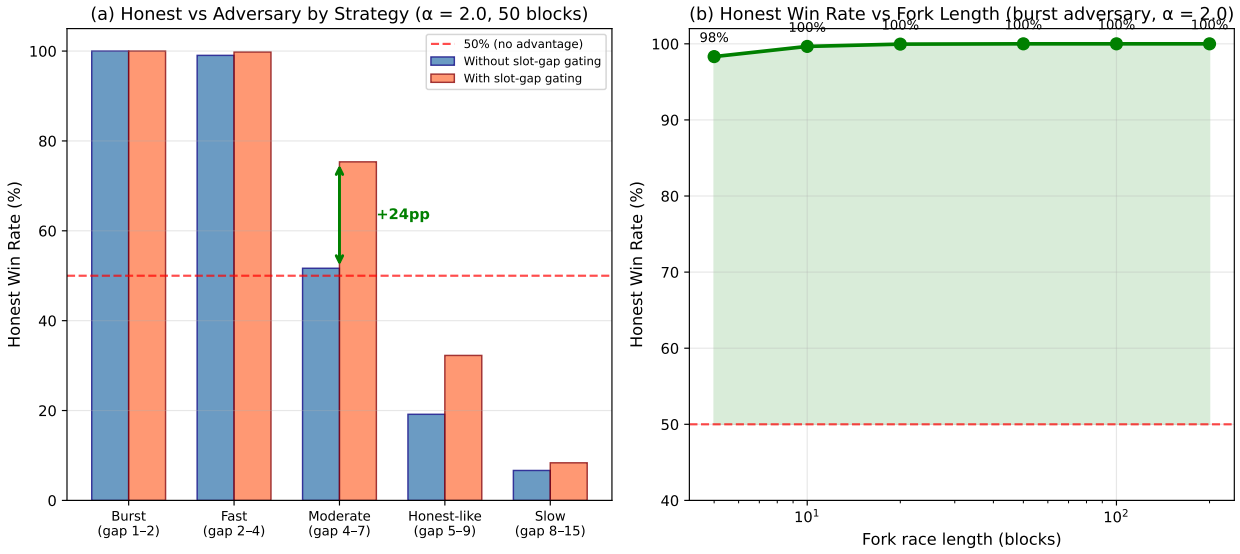


Figure 6: **Fork race results.** *(a)* Honest win rate by adversary strategy with $\alpha = 2$ over 50-block races. Blue bars: without gating (shifted exponential only). Orange bars: with slot-gap gating. The green arrow highlights the improvement at moderate gaps from 52% to 75%. Burst and fast adversaries are overwhelmingly defeated in both cases. *(b)* Honest win rate vs. fork length against a burst adversary with gating. Even at 5 blocks, honest wins 98% of races. At 50+ blocks, honest wins 100%.

All three use identical superblock parameters (shifted exponential thresholds, slot-gap gating, $\alpha = 2$). The *only* variable is the L0 threshold function.

The results (Table 3) confirm the thesis. Under Taktikos LDD, honest chains win 98.4% of races even against moderate-gap adversaries, with a 3.6× weight ratio. Under Praos's flat threshold, this collapses to 69.2% with a 1.3× ratio—barely above a coin flip. The weight ratio at burst gaps degrades from 101× (Taktikos) to 2.6× (Praos), a 39× reduction in security margin.

**Why the collapse occurs.** In Praos, $\varphi(\delta) = 0.032$ for all $\delta$. The weight term $\varphi(\delta)^\alpha$ becomes a constant $(0.032)^2 = 0.001$ that multiplies every block equally. The weight function reduces to:

$$W_{\text{Praos}}(C) = 0.001 \cdot \sum_{B \in C} \left(1 + \textstyle\sum_\mu 2^\mu \cdot \mathbf{1}[\text{B hits } \mu]\right)$$

This is a constant times the super-level-weighted block count. Since slot-gap gating still operates, burst-forging receives some super-level suppression—hence the 97.9% at burst gaps—but a moderate-gap adversary ($\delta = 4$–$7$) achieves gating factors of 0.27–0.47, close enough to honest (0.47–0.67) that stochastic variation dominates.

Under Taktikos LDD, the $\varphi(\delta)^2$ term provides an *additional* 47× discrimination that is independent of and multiplicative with the gating mechanism. This is the PoW analogy made precise: LDD gives each block a "difficulty stamp" that the weight function aggregates into cumulative security.

Figure 8 and Figure 9 visualize the comparison.

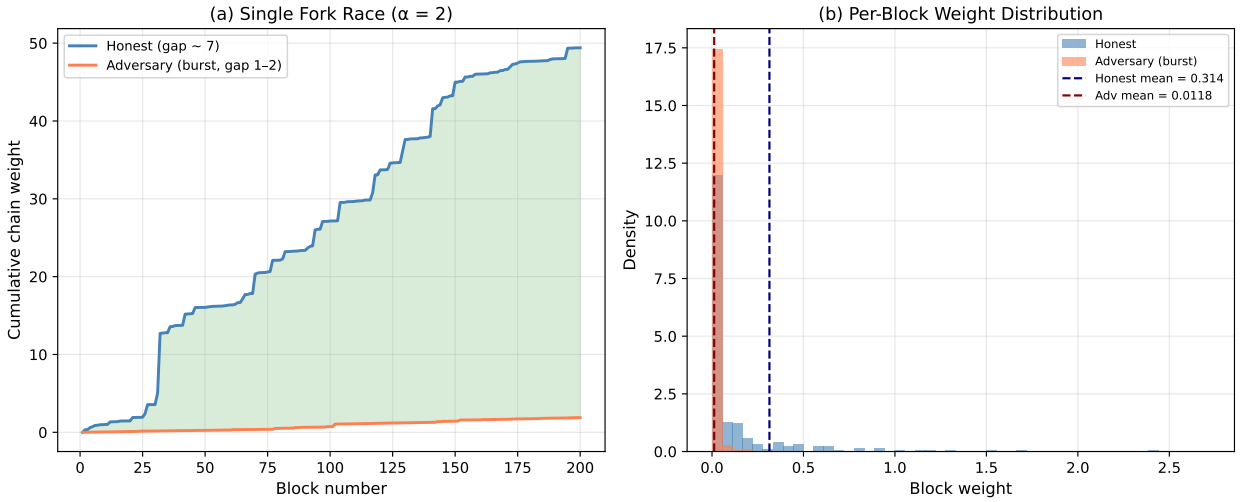


Figure 7: **Cumulative weight analysis.** *(a)* A single 200-block fork race. The honest chain (blue) steadily outpaces the burst adversary (red). The green fill shows the growing weight gap. Honest finishes at approximately 10× the adversary's cumulative weight. *(b)* Per-block weight distributions over 10,000 blocks. Honest blocks (blue) cluster around a higher mean, while adversary burst blocks (red) cluster near zero. The distributions have minimal overlap, meaning weight-based chain selection provides strong discrimination.

| Adversary Strategy | No Gating | | With Gating | |
|---|---|---|---|---|
| | Win % | Weight Ratio | Win % | Weight Ratio |
| Burst (gap 1–2) | 100.0 | 10.6× | 100.0 | 26.4× |
| Fast (gap 2–4) | 99.0 | 4.2× | 99.8 | 8.6× |
| Moderate (gap 4–7) | 51.7 | 1.1× | 75.3 | 1.6× |
| Honest-like (gap 5–9) | 19.2 | 0.7× | 32.3 | 0.9× |

Table 2: Fork race results: honest win percentage and weight ratio (honest cumulative weight / adversary cumulative weight) for 50-block races with $\alpha = 2$. Slot-gap gating strengthens honest advantage across all strategies, with the largest improvement at moderate gaps.

### 5.6 Light Client Overhead

For a chain of $N_0 \approx 1.43$M base blocks (10M slots at approximately 14.3% fill rate) with security parameter $m = 50$ and tip window $k = 100$:

- Level-9 subchain: approximately 2,300 headers
- Progressive fill to L0 near tip: approximately 2,700 additional
- Tip window: 100 L0 headers
- **Total: approximately 5,100 headers** vs. 1.43M full (approximately 280× reduction)

| | Taktikos LDD | | Mild LDD | | Praos Flat | |
|---|---|---|---|---|---|---|
| Adversary Strategy | Win% | Ratio | Win% | Ratio | Win% | Ratio |
| Burst (gap 1–2) | 100.0 | 101× | 99.6 | 5.5× | 97.9 | 2.6× |
| Fast (gap 2–4) | 100.0 | 17.8× | 98.6 | 3.3× | 91.8 | 1.9× |
| Moderate (gap 4–7) | 98.4 | 3.6× | 87.3 | 1.8× | 69.2 | 1.3× |
| Honest-like (gap 5–9) | 85.2 | 1.8× | 66.3 | 1.3× | 54.5 | 1.2× |

Table 3: Fork race results under three L0 threshold functions. All use identical superblock parameters with $\alpha = 2$. The Taktikos LDD snowplow provides a 7.7× threshold range between gap 1 and gap 7, which $\alpha = 2$ amplifies to a 47× per-block weight range. Praos's constant threshold collapses this range to 1.0×, reducing weight-based chain selection to approximate super-level counting. The degradation is monotonic: as the L0 difficulty gradient flattens, honest advantage erodes at every adversary strategy.

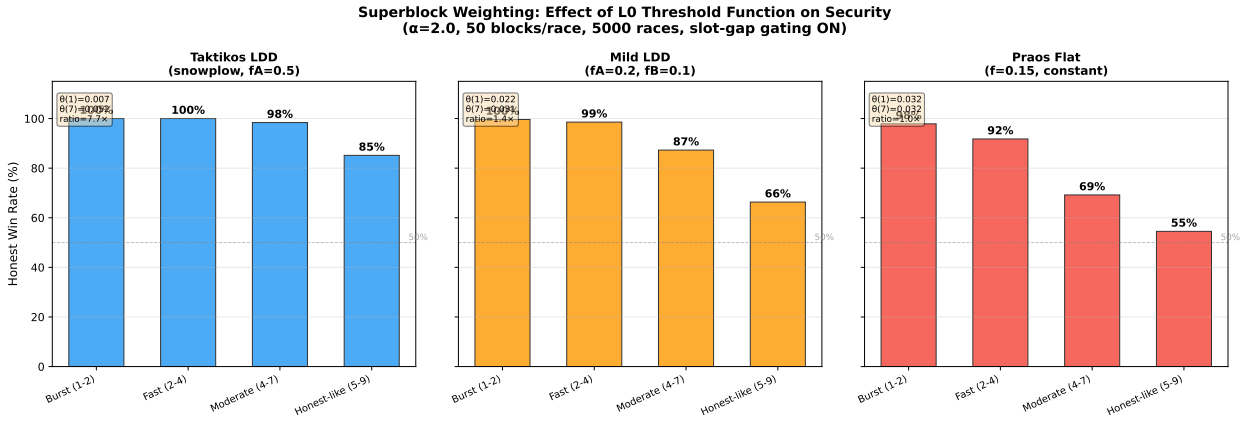


Figure 8: **Honest win rates under three L0 threshold functions.** Left: Taktikos LDD (7.7× threshold range) achieves near-perfect honest wins across all strategies. Center: Mild LDD (1.4× range) shows erosion at moderate and honest-like gaps. Right: Praos flat (1.0× range, no difficulty gradient) collapses to 69% at moderate gaps—weight-based selection provides only marginal advantage over length-based selection. The 50% line (dashed) represents a fair coin flip.

## 6 Discussion

**The honest-like adversary.** An adversary forging at gaps 5–9 wins 68% of races even with gating. This is expected: at honest-like gaps, the adversary's blocks are indistinguishable from honest blocks in both weight and super-level density. The defense is that such an adversary has no *rate advantage*—they produce blocks at the same pace as honest stakers and cannot overtake a chain of equal or greater length. The weight mechanism's role is to penalize adversaries who *do* attempt to gain a rate advantage through fast forging, not to detect adversaries who forge at honest rates.

**Choice of $\alpha$.** The threshold exponent $\alpha$ controls the discrimination power. At $\alpha = 1$, honest wins 98% against burst but only 53% against moderate adversaries. At $\alpha = 2$, these improve to 100% and 75%. Higher $\alpha$ further amplifies the gap advantage but increases weight variance, potentially making the system more susceptible to fortuitous high-gap blocks. The value $\alpha = 2$ is recommended

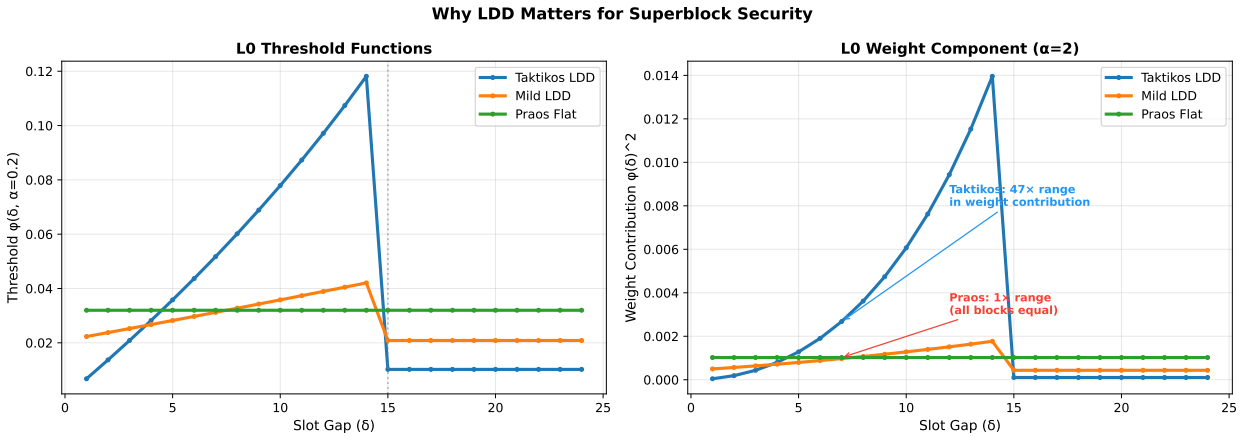


Figure 9: **L0 threshold functions and their weight contributions.** *(a)* Raw threshold values. Taktikos spans a wide range; Praos is a flat line. *(b)* After squaring ($\alpha = 2$), the Taktikos curve spans a 47× range in per-block weight contribution, while Praos contributes identical weight to every block. This is why superblock proofs require a difficulty gradient: without it, there is no notion of a block of greater difficulty to anchor the weight function.

as a practical balance.

**Independence vs. nesting.** This construction produces *independent* level assignments (a block may hit L3 without hitting L1), unlike PoW where $\mu$-superblock status implies $(\mu - 1)$-superblock status. Independence strengthens security by requiring the adversary to satisfy $L$ independent constraints simultaneously, and prevents a fortuitous high-level hit from automatically boosting all lower levels.

**Relationship to the Taktikos consistency bound.** The original Taktikos consistency bound has a known vulnerability at moderate slot gaps where the LDD curve's linear ramp provides insufficient discrimination. This weight construction addresses the vulnerability: it corresponds to the moderate-gap adversary (gap 4–7), where slot-gap gating provides the additional 23 percentage points of honest advantage that the LDD threshold alone cannot.

**Integration cost.** The construction requires: (1) $L-1$ additional hash evaluations per base block (negligible), (2) $48L$ bytes of header overhead for the subchain state vector, and (3) replacement of length-based chain selection with weight-based selection. The weight function is deterministically computable from header data, requiring no additional state.

**Long-range attacks.** Long-range attacks [14], in which an adversary uses expired stake to forge an alternative history, are a known concern for PoS light clients. In the Ouroboros family, key-evolving signatures (KES) [4] provide the primary mitigation: honest parties erase old signing keys, making it computationally infeasible to forge blocks at past time steps. This construction inherits the KES defense directly from the underlying protocol. Additionally, the LDD-weighted chain selection provides a secondary defense: a long-range adversary attempting to forge an alternative history must produce blocks with appropriate slot gaps to accumulate competitive weight. Forging with small gaps (to maximize chain length) results in negligible cumulative weight, while forging at honest-like gaps limits chain growth rate, compounding the difficulty of the attack.

**Strategic withholding.** Selfish mining [15], in which a staker strategically withholds blocks to gain advantage, is particularly difficult under LDD-weighted chain selection. Under the original Taktikos `maxvalid-tk` rule, a withheld block at a low slot gap may be superseded by a competing block at a higher slot gap (which the protocol prefers due to its lower head-slot number). Under the weighted extension presented here, withholding is further penalized: the withheld block's weight contribution is fixed at forging time, while the honest chain continues accumulating weight from naturally distributed slot gaps. The adversary cannot retroactively improve a withheld block's weight, and the delay in publication risks losing the chain race entirely. This creates a strict incentive for immediate publication, aligning with the "build fast and honestly" philosophy of LDD.

**Nothing at stake.** The classical nothing-at-stake objection [5]—that a rational staker should extend all competing chains since doing so is costless—is addressed at the protocol level by the Ouroboros family's slashing conditions and VRF-based private leader election (which prevents an adversary from knowing *a priori* which stakers are eligible). The weighted chain selection presented here does not introduce new nothing-at-stake vulnerabilities: the weight function is deterministic given the block headers, so a staker extending multiple chains cannot create "phantom weight." Each chain fork accumulates weight independently based on its actual production timing, and the heaviest chain wins deterministically.

# 7 Conclusion

NIPoPoWs were conceived as a PoW construction because proof-of-work provides a natural difficulty gradient: blocks with more leading zeros are exponentially rarer and carry more weight. It has been shown that local dynamic difficulty in proof-of-stake provides the same gradient. LDD was designed for throughput—but it simultaneously gives PoS something it previously lacked: blocks of varying difficulty.

This construction exploits the gradient through shifted exponential thresholds, slot-gap gating, and cumulative weight. The result is a PoS analog of Bitcoin's total-work security [8]: burst-forging produces blocks with near-zero weight (47× lower than honest), while forging at honest-like gaps yields no length advantage. Critically, this security *requires* LDD: under Praos's constant threshold, the weight function collapses and honest advantage erodes from 98% to 69% at moderate adversary gaps.

This reveals a previously unrecognized duality in the design of LDD: the same property that regularizes block production (throughput benefit) also creates a difficulty-based security model (superblock benefit). Future work includes formal security proofs in the Universal Composability framework, extension to variable stake distributions, and integration with recursive superblock proofs for cross-chain verification.

# References


[1] A. Kiayias, A. Miller, and D. Zindros. Non-interactive proofs of proof-of-work. In *Financial Cryptography and Data Security*, 2020.

[2] B. Bünz, L. Kiffer, L. Luu, and M. Zamani. FlyClient: Super-light clients for cryptocurrencies. In *IEEE Symposium on Security and Privacy*, 2020.

[3] P. Gaži, A. Kiayias, and A. Russell. Ouroboros Taktikos: Regularizing proof-of-stake via dynamic difficulty. In *Financial Cryptography and Data Security*, 2024.

[4] B. David, P. Gaži, A. Kiayias, and A. Russell. Ouroboros Praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In *EUROCRYPT*, 2018.

[5] A. Kiayias, A. Russell, B. David, and R. Oliynykov. Ouroboros: A provably secure proof-of-stake blockchain protocol. In *CRYPTO*, 2017.

[6] C. Badertscher, P. Gaži, A. Kiayias, A. Russell, and V. Zikas. Ouroboros Genesis: Composable proof-of-stake blockchains with dynamic availability. In *ACM CCS*, 2018.

[7] P. Chaidos and A. Kiayias. Mithril: Stake-based threshold multisignatures. In *ASIACRYPT*, 2024.

[8] J. Garay, A. Kiayias, and N. Leonardos. The Bitcoin backbone protocol: Analysis and applications. In *EUROCRYPT*, 2015.

[9] Y. Sompolinsky and A. Zohar. Secure high-rate transaction processing in Bitcoin. In *Financial Cryptography and Data Security*, 2015.

[10] S. Micali, M. Rabin, and S. Vadhan. Verifiable random functions. In *FOCS*, 1999.

[11] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. Algorand: Scaling Byzantine agreements for cryptocurrencies. In *SOSP*, 2017.

[12] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch. Verifiable delay functions. In *CRYPTO*, 2018.

[13] J. Long. Nakamoto consensus with verifiable delay puzzle. *arXiv preprint arXiv:1908.06394*, 2019.

[14] E. Deirmentzoglou, G. Papakyriakopoulos, and C. Patsakis. A survey on long-range attacks for proof of stake protocols. *IEEE Access*, 7:28712–28725, 2019.

[15] I. Eyal and E. G. Sirer. Majority is not enough: Bitcoin mining is vulnerable. In *Financial Cryptography and Data Security*, 2014.

[16] V. Buterin and V. Griffith. Casper the friendly finality gadget. *arXiv preprint arXiv:1710.09437*, 2017.

[17] A. Stewart and E. Kokoris-Kogia. GRANDPA: A Byzantine finality gadget. *arXiv preprint arXiv:2007.01560*, 2020.

[18] E. Buchman, J. Kwon, and Z. Milosevic. The latest gossip on BFT consensus. *arXiv preprint arXiv:1807.04938*, 2018.

[19] M. Al-Bassam, A. Sonnino, and V. Buterin. Fraud and data availability proofs: Maximising light client security and scaling blockchains with dishonest majorities. *arXiv preprint arXiv:1809.09044*, 2019.

[20] A. Kiayias and D. Zindros. Proof-of-work sidechains. In *Financial Cryptography and Data Security*, 2020.

[21] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.

# A Alternative Designs Explored

During development, several alternative parameterizations were explored:

- **Flat conditional probabilities** ($P_\mu = 1/2^\mu$): Correct density but zero security advantage. Adversary obtains super hits proportional to base chain length, making weight a deterministic function of length.
- **Per-level LDD with slot-gap tracking**: Extended the L0 snowplow to super levels with cutoffs $\gamma_\mu = \gamma \cdot 2^\mu$. Failed because super levels are L0-gated (approximately 15% of slots), causing slot-based gaps to grow disproportionately and collapsing all super levels to baseline rates.
- **Cascading dormant periods** ($\psi_\mu = \gamma_{\mu-1}$): Addressed the shape but achieved only approximately 50% of target hit rates due to aggressive dormant zones.
- **Block-count gap LDD for high levels**: Inverted the expected ordering—higher levels converged to baseline before lower levels.
- **Overage-ratio weighting** ($w_\mu = 2^\mu \cdot E[g_\mu]/g_\mu$): Initially rewarded burst-forging because inverse-gap weighting gives high values at small gaps. Replaced with threshold-based weighting.

The final construction (shifted exponential + slot-gap gating + threshold weighting) emerged from systematically addressing each failure mode.

# B Simulation Implementation

The simulation is implemented in Scala 2.13 using Cats Effect. VRF uses Ed25519VRF (ECVRF-ED25519-SHA512-TAI per draft-irtf-cfrg-vrf-04) backed by `curve25519-elisabeth`. Fork race experiments use Python with SHA-256 hash simulation.